8.5 MULTILEVEL INHERITANCE

It is not uncommon that a class is derived from another derived class as shown in Fig. 8.7. The class **A** serves as a base class for the derived class **B**, which in turn serves as a base class for the derived class C. The class **B** is known as *intermediate* base class since it provides a link for the inheritance between **A** and **C**. The chain **ABC** is known as inheritance *path*.

A derived class with multilevel inheritance is declared as follows:

```
class A{.....};          // Base class
class B: public A {.....};     // B derived from A
class C: public B {.....};     // C derived from B
```

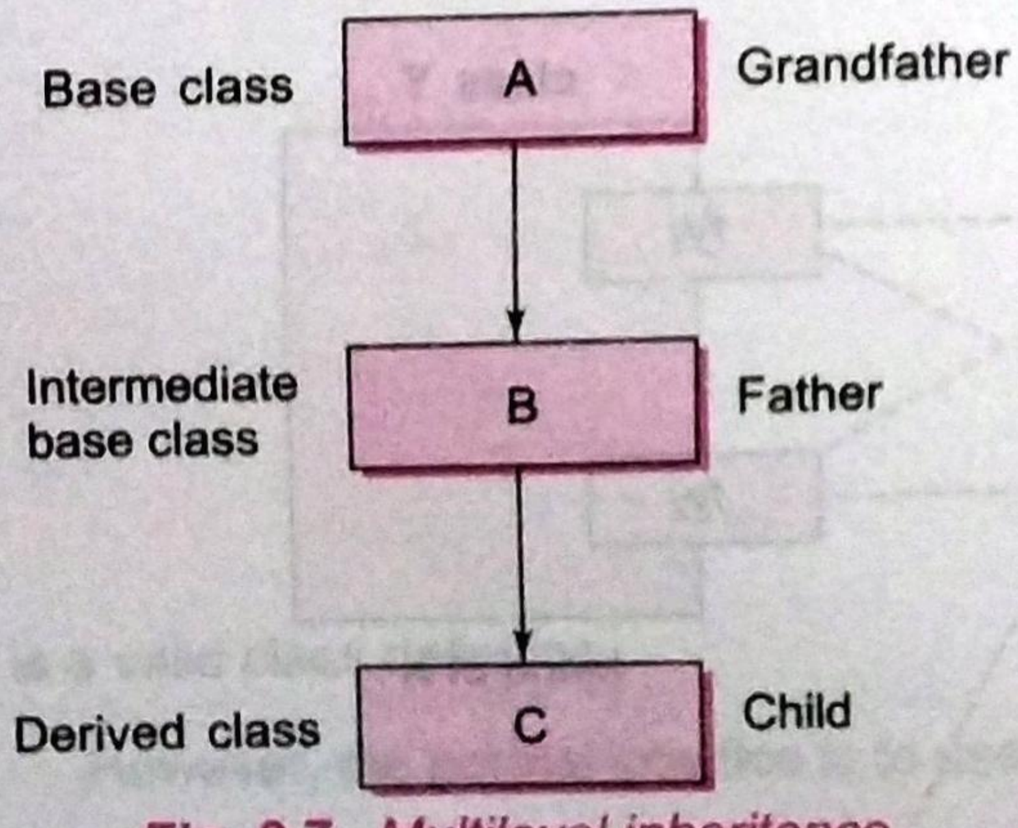

Fig. 8.7 *Multilevel inheritance*

This process can be extended to any number of levels.

Let us consider a simple example. Assume that the test results of a batch of students are stored in three different classes. Class **student** stores the roll-number, class **test** stores the marks obtained in two subjects and class **result** contains the **total** marks obtained in the test. The class **result** can inherit the details of the marks obtained in the test and the roll-number of students through multilevel inheritance.
Example:

The class **result**, after inheritance from 'grandfather' through 'father', would contain the following members:

```
private:
    float total;                                    // own member
protected:
    int roll_number;                                // inherited from student
                                                    // via test
    float sub1;                                     // inherited from test
    float sub2;                                     // inherited from test
public:
    void get_number(int);                           // from student via test
    void put_number(void);                          // from student via test
    void get_marks(float, float);                   // from test
    void put_marks(void);                           // from test
    void display(void);                             // own member
```

Program 8.3 Multilevel Inheritance

```
#include <iostream>

using namespace std;

class student
{
    protected:
        int roll_number;
    public:
        void get_number(int);
        void put_number(void);
};

void student :: get_number(int a)
{
        roll_number = a;
}
void student :: put_number()
{
        cout << "Roll Number: " << roll_number << "\n";
}
class test : public student          // First level derivation
{
protected:
    float sub1;
```

```
        float sub2;
    public:
        void get_marks(float, float);
        void put_marks(void);
};

void test :: get_marks(float x, float y)
{
        sub1 = x;
        sub2 = y;
}

void test :: put_marks()
{
        cout << "Marks in SUB1 = " << sub1 << "\n";
        cout << "Marks in SUB2 = " << sub2 << "\n";
}

class result : public test          // Second level derivation
{
        float total;                // private by default
    public:
        void display(void);
};

void result :: display(void)
{
        total = sub1 + sub2;
        put_number();
        put_marks();
        cout << "Total = " << total << "\n";
}
int main()
{
        result student1;            // student1 created

        student1.get_number(111);

        student1.get_marks(75.0, 59.5);

        student1.display();

        return 0;
}
```

The output of Program 8.3 would be:

```
Roll Number: 111
Marks in SUB1 = 75
Marks in SUB2 = 59.5
Total = 134.5
```

8.6 MULTIPLE INHERITANCE

A class can inherit the attributes of two or more classes as shown in Fig. 8.8. This is known as *multiple inheritance*. Multiple inheritance allows us to combine the features of several existing classes as a starting point for defining new classes. It is like a child inheriting the physical features of one parent and the intelligence of another.

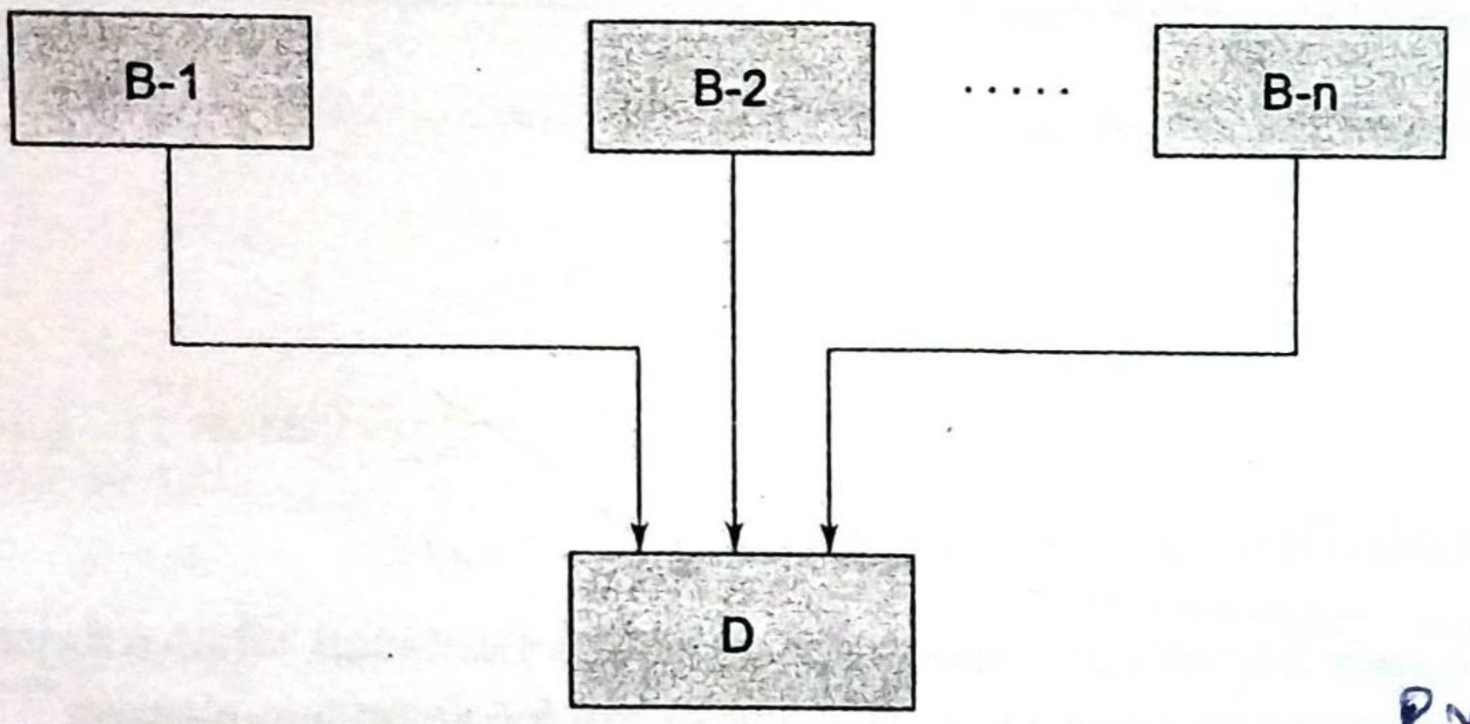

Fig. 8.8 *Multiple inheritance*

Prog Next Pg

The syntax of a derived class with multiple base classes is as follows:

```
class D: visibility B-1, visibility B-2 ...
{
     .....
     ..... (Body of D)
     .....
};
```

where, *visibility* may be either **public** or **private**. The base classes are separated by commas.

Example:

```
class P : public M, public N
{
     public:
          void display(void);
};
```

Program 8.4 Multiple Inheritance

```
#include <iostream>

using namespace std;

class M
{
    protected:
        int m;
    public:
        void get_m(int);
};

class N
{
    protected:
        int n;
    public:
        void get_n(int);
};

class P : public M, public N
{
    public:
        void display(void);
};

void M :: get_m(int x)
{
        m = x;
}

void N :: get_n(int y)
{
        n = y;
}

void P :: display(void)
{
        cout << "m = " << m << "\n";
        cout << "n = " << n << "\n";
        cout << "m*n = " << m*n << "\n";
}
int main()
{
        P p;
```

object of 3rd class
is class P

```
        p.get_m(10);
        p.get_n(20);
        p.display();

        return 0;
}
```

The output of Program 8.4 would be:

```
m = 10
n = 20
m*n = 200
```

Ambiguity Resolution in Inheritance

Occasionally, we may face a problem in using the multiple inheritance, when a function with the same name appears in more than one base class. Consider the following two classes.

```
class M
{
    public:
        void display(void)
        {
            cout << "Class M\n";
        }
};

class N
{
    public:
        void display(void)
        {
            cout << "Class N\n";
        }
};
```

Which **display()** function is used by the derived class when we inherit these two classes? We can solve this problem by defining a named instance within the derived class, using the class resolution operator with the function as shown below:

```
class P : public M, public N
{
    public:
        void display(void)      // overrides display() of M and N
        {
            M :: display(); }
        };
```

We can now use the derived class as follows:

```
int main()
{
        P p;
        p.display();
}
```

Ambiguity may also arise in single inheritance applications: For instance, consider the following situation:

```
class A
{
    public:
        void display()
        {
            cout << "A\n";
        }
};
class B : public A
{
    public:
        void display()
        {
            cout << "B\n";
        }
};
```

In this case, the function in the derived class overrides the inherited function and, therefore, a simple call to **display()** by **B** type object will invoke function defined in **B** only. However, we may invoke the function defined in **A** by using the scope resolution operator to specify the class.

Example:

```
int main()
{
        B b;
        b.display();          // derived class object
        b.A::display();       // invokes display() in B
        b.B::display();       // invokes display() in A
                              // invokes display() in B
        return 0;
}
```

This will produce the following output:

```
B
A
B
```

8.8 HYBRID INHERITANCE

There could be situations where we need to apply two or more types of inheritance to design a program. For instance, consider the case of processing the student results discussed in Sec. 8.5. Assume that we have to give weightage for sports before finalising the results. The weightage for sports is stored in a separate class called **sports**. The new inheritance relationship between the various classes would be as shown in Fig. 8.11.

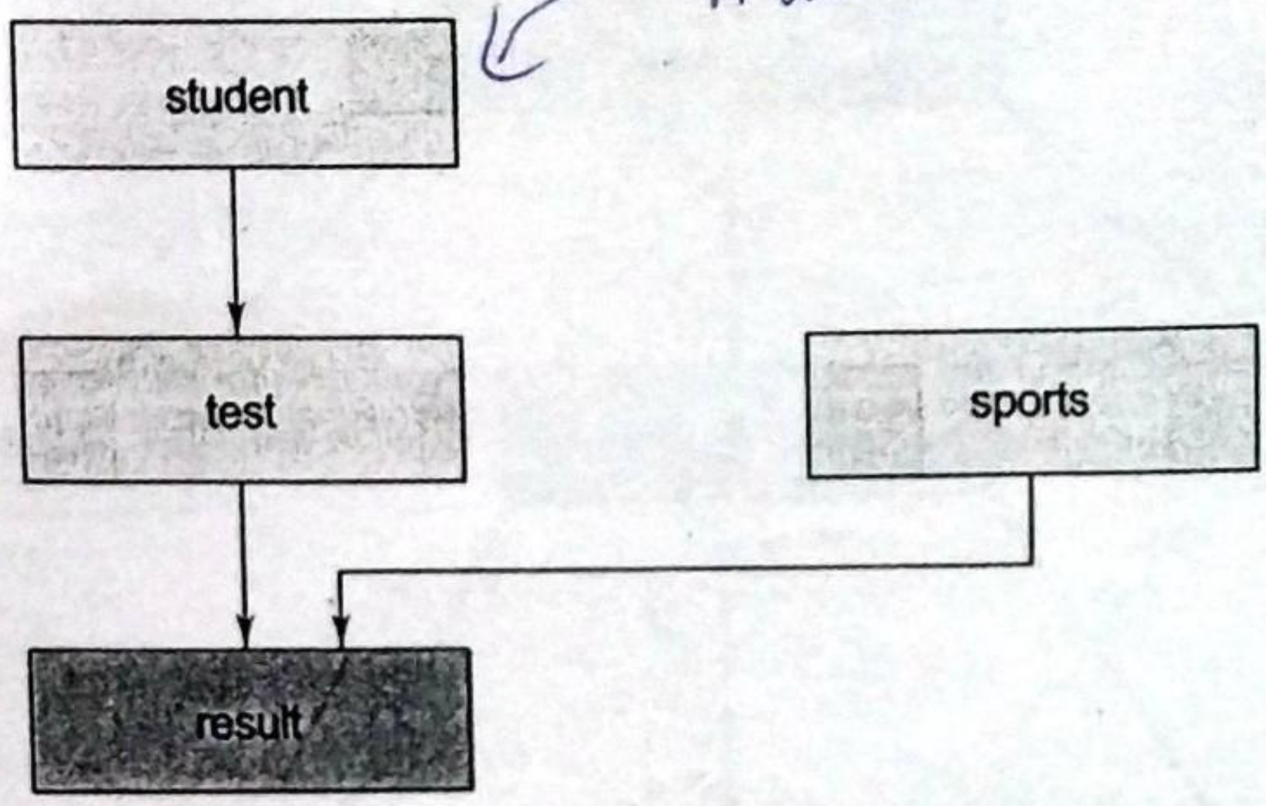

Fig. 8.11 *Multilevel, multiple inheritance*

The **sports** class might look like:

```
class sports
{
  protected:
    float score;
  public:
    void get_score(float);
    void put_score(void);
};
```

The result will have both the multilevel and multiple inheritances and its declaration would be as follows:

```
class result : public test, public sports
{
    .....
    .....
};
```

Where test itself is a derived class from student. That is

```
class test : public student
{
    .....
    .....
};
```

Program 8.5 illustrates the implementation of both multilevel and multiple inheritance.

Program 8.5 Hybrid Inheritance

```
#include <iostream>
using namespace std;
class student
{
    protected:
        int roll_number;
    public:
        void get_number(int a)
        {
            roll_number = a;
        }
        void put_number(void)
        {
            cout << "Roll No: " << roll_number << "\n";
        }
};
class test : public student
{
    protected:
        float part1, part2;
    public:
        void get_marks(float x, float y)
        {
            part1 = x; part2 = y;
        }
        void put_marks(void)
        {
            cout << "Marks obtained: " << "\n"
                 << "Part1 = " << part1 << "\n"
                 << "Part2 = " << part2 << "\n";
        }
};
```

```
class sports
{
    protected:
        float score;
    public:
        void get_score(float s)
        {
            score = s;
        }
        void put_score(void)
        {
            cout << "Sports wt: " << score << "\n\n";
```

```
        }
};

class result : public test, public sports
{
        float total;
    public:
        void display(void);
};
void result :: display(void)
{
        total = part1 + part2 + score;

        put_number();
        put_marks();
        put_score();

        cout << "Total Score: " << total << "\n";
}

int main()
{
        result student_1;
        student_1.get_number(1234);
        student_1.get_marks(27.5, 33.0);
        student_1.get_score(6.0);
        student_1.display();

        return 0;
}
```

The output of Program 8.5 would be:

```
Roll No: 1234
Marks obtained:
Part1 = 27.5
Part2 = 33
Sports wt: 6

Total Score: 66.5
```